

Final Exam Project

Cryptographic Security Suite

Course: MAT364 - Cryptography

Instructor: Adil Akhmetov

University: SDU

Total Points: 40 points (40% of final grade)

Team Size: 3 students per group

Press Space for next page →

Project Overview

Objective

Build a **comprehensive cryptographic security suite** that demonstrates mastery of all major cryptographic concepts covered in this course, from classical ciphers to modern blockchain applications.

Project Scope

This is a **major project** requiring significant effort from all team members. You will create an integrated application that showcases:

- Classical and modern encryption
- Hash functions and data integrity
- Digital signatures and PKI
- Secure authentication
- Blockchain fundamentals

Learning Outcomes

Upon completion, students will demonstrate:

- Deep understanding of cryptographic primitives
- Ability to integrate multiple security mechanisms
- Secure programming practices
- System design and architecture skills
- Professional documentation and presentation

Project Description

The CryptoVault Suite

Concept

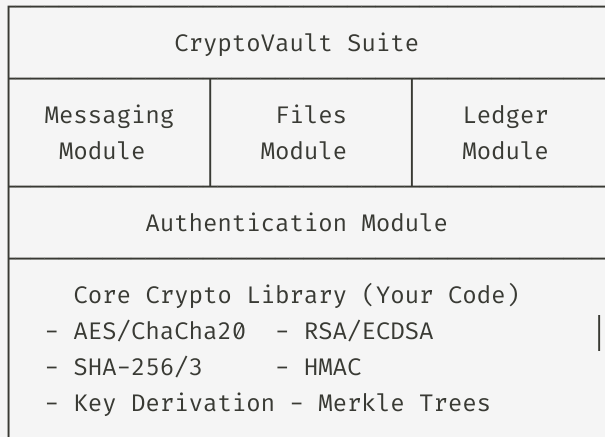
Build **CryptoVault** - a comprehensive cryptographic toolkit that provides:

1. **Secure Messaging Module** - End-to-end encrypted communication
2. **File Encryption Module** - Secure file storage and sharing
3. **Authentication Module** - Multi-factor secure login
4. **Blockchain Ledger Module** - Immutable audit trail

Real-World Scenario

Your team has been hired by a startup to create a security suite for protecting sensitive corporate communications and documents with an immutable audit trail.

System Architecture



Module 1: Authentication System (10 points)

Required Features (7 points)

User Registration (2 pts)

- Secure password hashing with **Argon2id** or **bcrypt**
- Salt generation using CSPRNG
- Password strength validation
- Store hashed password + salt

User Login (2 pts)

- Constant-time password verification
- Rate limiting (basic protection)
- Session token generation using **HMAC-SHA256**
- Secure session storage

Multi-Factor Auth (3 pts)

- **TOTP** (Time-based One-Time Password) implementation
- QR code generation for authenticator apps
- Backup codes with secure storage
- TOTP verification with time window tolerance

Implementation Example

```
class AuthModule:
    def register(self, username: str, password: str) -> dict:
        # 1. Validate password strength
        if not self.validate_password_strength(password):
            raise ValueError("Password too weak")

        # 2. Generate salt and hash password
        salt = secrets.token_bytes(32)
        password_hash = argon2.hash(password, salt)

        # 3. Generate TOTP secret
        totp_secret = pyotp.random_base32()

        # 4. Store user data
```

Bonus Features (+1 pt each, max 2)

- Password reset with secure token
- Account lockout after failed attempts

Module 2: Secure Messaging (10 points)

Required Features (8 points)

Key Exchange (2 pts)

- Implement **ECDH** (Elliptic Curve Diffie-Hellman) using P-256
- Generate ephemeral key pairs per session
- Derive shared secret using HKDF

Message Encryption (3 pts)

- **AES-256-GCM** for authenticated encryption
- Unique nonce/IV per message
- Message format: [nonce || ciphertext || auth_tag]

Digital Signatures (3 pts)

- **ECDSA** or **Ed25519** signatures on messages
- Sign message hash (SHA-256)
- Verify sender authenticity
- Non-repudiation for sent messages

Implementation Example

```
class MessagingModule:
    def __init__(self, private_key):
        self.private_key = private_key
        self.public_key = private_key.public_key()

    def send_message(self, recipient_pubkey: bytes,
                    message: str) → dict:
        # 1. ECDH key exchange
        shared_key = self.ecdh_derive(recipient_pubkey)

        # 2. Derive encryption key using HKDF
        enc_key = HKDF(shared_key, salt=os.urandom(16),
                       info=b"message_key")
```

Bonus Features (+1 pt each, max 2)

- Perfect Forward Secrecy with ratcheting
- Group messaging with shared keys

Module 3: File Encryption System (10 points)

Required Features (8 points)

File Encryption (3 pts)

- **AES-256-GCM** or **ChaCha20-Poly1305**
- Streaming encryption for large files
- Random file encryption key (FEK)
- FEK encrypted with user's master key

Key Derivation (2 pts)

- **PBKDF2** or **Argon2** from user password
- Minimum 100,000 iterations for PBKDF2
- Unique salt per encryption
- Key stretching documentation

Integrity Verification (3 pts)

- **SHA-256** hash of original file
- **HMAC-SHA256** for file authenticity
- Verify integrity before decryption
- Tamper detection and reporting

Implementation Example

```
class FileEncryptionModule:
    def encrypt_file(self, filepath: str,
                    password: str) → dict:
        # 1. Derive master key from password
        salt = os.urandom(32)
        master_key = hashlib.pbkdf2_hmac(
            'sha256', password.encode(), salt,
            iterations=100000, dklen=32
        )

        # 2. Generate file encryption key
        fek = os.urandom(32)

        # 3. Read and encrypt file (streaming)
```

Bonus Features (+1 pt each, max 2)

- Secure file sharing with RSA-OAEP
- Metadata encryption (filename hiding)

Module 4: Blockchain Audit Ledger (10 points)

Required Features (8 points)

Block Structure (2 pts)

- Previous block hash (SHA-256)
- Merkle root of transactions
- Timestamp and nonce
- Block hash meeting difficulty target

Merkle Tree (3 pts)

- Build tree from transaction hashes
- Generate Merkle proofs
- Verify transaction inclusion
- Handle odd number of leaves

Proof of Work (3 pts)

- Simple PoW with adjustable difficulty
- Find nonce where hash < target
- Block validation
- Chain integrity verification

Implementation Example

```
class BlockchainModule:
    def __init__(self, difficulty: int = 4):
        self.chain = []
        self.pending_transactions = []
        self.difficulty = difficulty

    def create_block(self, transactions: list) → dict:
        # 1. Build Merkle tree
        merkle_root = self.build_merkle_tree(transactions)

        # 2. Create block structure
        block = {
            'index': len(self.chain),
            'timestamp': int(time.time()),
```

Bonus Features (+1 pt each, max 2)

- Transaction signatures verification
- Chain reorganization handling

Audit Trail Integration

Logging Requirements

Every security-relevant action must be logged to the blockchain:

Authentication Events

```
{
  'type': 'AUTH_LOGIN',
  'user_hash': sha256(username), # Privacy
  'timestamp': unix_timestamp,
  'success': True/False,
  'ip_hash': sha256(ip_address)
}
```

File Operations

```
{
  'type': 'FILE_ENCRYPT',
  'file_hash': sha256(original_file),
  'user_hash': sha256(username),
  'timestamp': unix_timestamp,
  'encrypted_hash': sha256(encrypted_file)
}
```

Integration Example

```
class CryptoVault:
    def __init__(self):
        self.auth = AuthModule()
        self.messaging = MessagingModule()
        self.files = FileEncryptionModule()
        self.ledger = BlockchainModule(difficulty=4)

    def login(self, username: str, password: str,
              totp: str) → str:
        try:
            token = self.auth.login(username, password, totp)

            # Log to blockchain
            self.log_event({
```

Technical Requirements

Core Crypto Library Requirements

Must Implement From Scratch

At least **3** of the following must be implemented without using library functions for the core algorithm:

Option A: Classical Ciphers

- ☐ Caesar cipher with frequency analysis breaker
- ☐ Vigenère cipher with Kasiski examination

Option B: Hash Functions

- ☐ SHA-256 (simplified version acceptable)
- ☐ Merkle tree with proof generation

Option C: Symmetric Encryption

- ☐ AES key expansion (rounds can use library)
- ☐ XOR-based stream cipher with LFSR

Option D: Asymmetric Operations

- ☐ RSA key generation (prime finding)
- ☐ Modular exponentiation with square-and-multiply

May Use Library Functions

For production-quality modules, use established libraries:

```
# Recommended Python Libraries
from cryptography.hazmat.primitives import hashes
from cryptography.hazmat.primitives.asymmetric import ec, rsa
from cryptography.hazmat.primitives.ciphers import Cipher, algorithms, modes
from cryptography.hazmat.primitives.kdf.hkdf import HKDF
from cryptography.hazmat.primitives.kdf.pbkdf2 import PBKDF2

# For TOTP
import pyotp

# For password hashing
import argon2
# or
import bcrypt
```

Security Requirements

- ☒ CSPRNG for all random values (`secrets` module)
- ☒ Constant-time comparisons for sensitive data
- ☒ Proper key management (no hardcoded keys)
- ☒ Input validation on all user inputs

Code Quality Requirements

Structure Requirements

```
cryptovault/
├── README.md           # Project overview
├── requirements.txt     # Dependencies
├── setup.py            # Package setup
├── .gitignore
├── src/
│   ├── __init__.py
│   ├── main.py         # Entry point / CLI
│   └── auth/
│       ├── __init__.py
│       ├── registration.py
│       ├── login.py
│       └── totp.py
```

Documentation Standards

Code Documentation

```
def encrypt_message(self, recipient_pubkey: bytes,
                    message: str) → dict:
    """
    Encrypt a message for a recipient using hybrid encrypt.

    This function performs:
    1. ECDH key exchange to derive shared secret
    2. HKDF to derive AES key from shared secret
    3. AES-256-GCM encryption of the message
    4. ECDSA signature on the ciphertext

    Args:
        recipient_pubkey: Recipient's ECDSA public key (bytes)
        message: Plaintext message to encrypt (str)
```

Testing Requirements

- Minimum **70%** code coverage
- Unit tests for all cryptographic functions
- Integration tests for module interactions
- Security test cases (invalid inputs, tampering)

Grading Rubric

Detailed Grading (40 points total)

Module Implementation (28 points)

Authentication Module (10 pts)

Component	Points
Password hashing (Argon2/bcrypt)	2
Secure login with rate limiting	2
TOTP implementation	3
Session management	2
Bonus features (max 1)	1

Messaging Module (10 pts)

Component	Points
-----------	--------

File Encryption Module (10 pts)

Component	Points
AES-GCM file encryption	3
PBKDF2/Argon2 key derivation	2
SHA-256 + HMAC integrity	3
Bonus features (max 2)	2

Blockchain Module (10 pts)

Component	Points
Block structure	2
Merkle tree + proofs	3

Grading Rubric (continued)

Code Quality & Documentation (6 points)

Code Quality (3 pts)

Criteria	Points
Clean, modular code structure	1
Proper error handling	1
Secure coding practices	1

Documentation (3 pts)

Criteria	Points
README with setup instructions	0.5
Code comments and docstrings	1

Testing & Presentation (6 points)

Testing (2 pts)

Criteria	Points
Unit tests (>70% coverage)	1
Integration tests	0.5
Security test cases	0.5

Presentation & Defense (4 pts)

Criteria	Points
Clear system overview	1
Live demonstration	1

Grade Distribution Summary

Points Breakdown

Category	Max Points	Weight
Authentication Module	10	25%
Messaging Module	10	25%
File Encryption Module	10	25%
Blockchain Module	10	25%
Code Quality	3	7.5%
Documentation	3	7.5%
Testing	2	5%
Presentation	4	10%

Team Organization

Team Roles & Responsibilities

Suggested Role Distribution

Role A: Authentication & Security Lead

- Design and implement authentication module
- Implement password hashing and TOTP
- Ensure secure coding practices across project
- Write security analysis document
- Review code for vulnerabilities

Role B: Cryptography & Messaging Lead

- Implement core crypto library (from scratch parts)
- Design and implement messaging module
- Implement key exchange and signatures
- Handle file encryption module
- Ensure proper key management

Role C: Blockchain & Integration Lead

- Design and implement blockchain module
- Create Merkle tree implementation
- Integrate all modules together
- Implement audit logging

Collaboration Requirements

All Members Must:

- Understand all cryptographic concepts used
- Be able to explain any code in the project
- Participate in code reviews
- Contribute to documentation
- Present during the defense

Git Workflow

```
# Each member works on feature branches
git checkout -b feature/auth-module

# Regular commits with meaningful messages
git commit -m "Implement TOTP verification with time window"

# Pull requests for code review
# At least 1 approval required before merge

# Main branch always deployable
git checkout main
git merge --no-ff feature/auth-module
```

Individual Accountability

Contribution Tracking

GitHub Contributions

- Commits should be distributed among all members
- Each member should have meaningful commits
- Code should be attributed to actual authors

Peer Evaluation

After submission, each team member will submit a confidential peer evaluation:

```
Team Member: [Name]
Contribution Level: [1-5]
Areas Contributed:
- [ ] Authentication
- [ ] Messaging
- [ ] File Encryption
- [ ] Blockchain
- [ ] Testing
- [ ] Documentation

Comments: [Optional feedback]
```

Defense Questions

During the presentation, each member will be asked:

General Questions

- Explain how [specific algorithm] works
- Why did you choose [specific approach]?
- What are the security implications of [design decision]?

Code-Specific Questions

- Walk through this function line by line
- What happens if [edge case] occurs?
- How does this prevent [specific attack]?

Grade Adjustment

Individual grades may be adjusted based on:

- Peer evaluations
- Git commit history
- Defense performance
- Demonstrated understanding

When self-evaluating, be honest and realistic about your own contributions.

Deliverables & Timeline

Submission Requirements

Required Deliverables

1. GitHub Repository

- ☐ Public repository with all source code
- ☐ Clean commit history from all members
- ☐ Properly structured as specified
- ☐ No sensitive data (keys, passwords)

2. Documentation Package

- ☐ **README.md** - Setup and usage instructions
- ☐ **architecture.md** - System design with diagrams
- ☐ **security_analysis.md** - Threat model and mitigations
- ☐ **user_guide.md** - How to use each module

3. Working Application

- ☐ All 4 modules functional
- ☐ CLI or simple UI interface
- ☐ Demo data/scenarios prepared
- ☐ Tests passing

4. Presentation

Project Duration: 5 Days

This is an intensive **5-day final project**. Plan your time wisely:

Day	Focus
Day 1	Team setup, repo creation, core crypto library
Day 2	Authentication + Messaging modules
Day 3	File Encryption + Blockchain modules
Day 4	Integration, testing, documentation
Day 5	Final polish + Defense presentation

Submission Deadline

All materials due: Day 5, 23:59 **Defense presentations:** Day 5

Presentation Format

Presentation Structure (15-20 min)

1. Introduction (2 min)

- Team introduction
- Project overview
- Problem statement

2. Architecture (3 min)

- System design
- Module interactions
- Technology choices

3. Cryptographic Deep Dive (5 min)

- From-scratch implementations
- Security decisions
- Key algorithms explained

4. Live Demo (5 min)

- User registration with TOTP
- Secure message exchange
- File encryption/decryption

Demo Scenario

Prepare a scripted demo showing:

DEMO SCRIPT

1. USER REGISTRATION

- Alice registers with strong password
- Shows TOTP QR code setup
- Stores credentials securely

2. LOGIN WITH MFA

- Alice logs in with password + TOTP
- Session token generated
- Login event logged to blockchain

3. SECURE MESSAGING

Backup Plan

Record video backup in case live demo fails

Security Analysis Requirements

Threat Model Document

Required Sections

1. Assets Identification

What are you protecting?

- User credentials
- Encryption keys
- Message contents
- File contents
- Audit logs

2. Threat Actors

Who might attack?

- External attackers
- Malicious insiders
- Compromised systems

3. Attack Vectors

How might they attack?

- Network interception

4. Security Measures

How do you defend?

Threat	Mitigation
Password brute force	Argon2id + rate limiting
MITM on messages	ECDH + signatures
File tampering	HMAC + blockchain log
Key theft	Key derivation, no storage
Replay attacks	Nonces, timestamps

5. Known Limitations

Be honest about weaknesses:

- No HSM support
- Single-machine deployment

Common Mistakes to Avoid

Security Mistakes

❌ Hardcoded keys or secrets

```
# BAD
SECRET_KEY = "mysecretkey123"

# GOOD
SECRET_KEY = os.environ.get('SECRET_KEY')
```

❌ Using random instead of secrets

```
# BAD
import random
key = random.randbytes(32)

# GOOD
import secrets
key = secrets.token_bytes(32)
```

❌ Non-constant time comparison

```
# BAD
if user_hash == stored_hash:
```

Implementation Mistakes

❌ No error handling

```
# BAD
ciphertext = cipher.encrypt(data)

# GOOD
try:
    ciphertext = cipher.encrypt(data)
except Exception as e:
    logger.error(f"Encryption failed: {e}")
    raise EncryptionError("Failed to encrypt data")
```

❌ Logging sensitive data

```
# BAD
logger.info(f"User {username} password: {password}")

# GOOD
logger.info(f"User {username} login attempt")
```

❌ Missing input validation

```
# BAD
```

Resources & Support

Documentation Resources

Cryptographic Libraries

- [Python cryptography docs](#)
- [PyCryptodome docs](#)
- [PyOTP for TOTP](#)

Standards & References

- [NIST Cryptographic Standards](#)
- [RFC 6238 - TOTP](#)
- [RFC 5869 - HKDF](#)

Books

- "Real-World Cryptography" - David Wong
- "Serious Cryptography" - Jean-Philippe Aumasson
- "Cryptography Engineering" - Ferguson, Schneier

Getting Help

Office Hours

- **When:** [TBD]
- **Where:** [TBD]
- **Email:** adil.akhmetov@sdu.edu.kz

FAQ

Q: Can we use a web framework? A: Yes, Flask/FastAPI are allowed for UI, but the crypto must be your implementation.

Q: What if we can't finish all modules? A: Prioritize quality over quantity. A well-implemented subset is better than buggy full implementation.

Q: Can we use different languages? A: Python is strongly recommended. Other languages allowed with instructor approval.

Q: How do we handle key storage? A: For this project, file-based storage with encryption is acceptable. Document the limitations.

Academic Integrity

Allowed

- ✓ Using cryptographic libraries as documented
- ✓ Referencing course materials and lectures
- ✓ Using online documentation and tutorials
- ✓ Collaborating within your team
- ✓ Asking instructor for clarification
- ✓ Using AI tools for understanding concepts

Not Allowed

- ✗ Copying code from other teams
- ✗ Sharing code with other teams
- ✗ Copying code without attribution
- ✗ Submitting code you don't understand
- ✗ Having someone outside the team write code
- ✗ Using AI to generate entire implementations without understanding

Attribution Requirements

When using external code:

```
# From: https://stackoverflow.com/questions/12345
# Author: username
# Modified: Added error handling
def some_function():
    pass
```

Consequences of Violations

- **First offense:** Zero on project, academic warning
- **Repeat offense:** Course failure, disciplinary action

The Golden Rule

If you can't explain every line of code in your submission during the defense, you shouldn't submit it.

Start Building!

Form your teams, set up your repositories, and start coding!

5-Day Final Project:

- Day 1: Setup & Core Crypto
- Day 2-3: Module Implementation
- Day 4: Integration & Testing
- Day 5: Submission & Defense

Questions? Contact: adil.akhmetov@sdu.edu.kz 